

Chapter 3

Classical Symmetric- Key Ciphers



Outlines

1. Classical Substitution Ciphers.
2. Classical Transposition / Permutation Ciphers.
3. Product Ciphers.

Learning Objectives

In the end of this Chapter 3, students should be able:

1. To understand and implement **classical substitution techniques**, including monoalphabetic and polyalphabetic substitution ciphers to encrypt a message.
2. To understand and implement **classical transposition or permutation techniques**, including key and keyless transposition ciphers to encrypt a message.
3. To understand the concept of **product cipher** in encryption.
4. To understand and perform **cryptanalysis** to the above classical techniques in different perspectives.

Classical Symmetric-Key Ciphers (**Overview**)

- The classical (or traditional) symmetric-key ciphers can be divided into **two broad** categories:
 1. **Substitution ciphers.**
 - It **replaces** one symbol with another symbol.
 - It can be categorized further into **monoalphabetic** and **polyalphabetic ciphers**.
 2. **Transposition / Permutation ciphers.**
 - It **changes** or **reorder** the position of the symbols.
 - It can be done either **keyless**, or **with a key**.

SUBSTITUTION CIPHERS

Substitution Cipher – Overview

- A technique that **replaces** the letters in plaintext by **other letters / numbers / symbols**.
- If the plaintext is viewed as a sequence of **bits**, then it involves replacing the plaintext bit patterns with ciphertext bit patterns.
- Substitution technique can be further divided into **two categories**:
 1. **Monoalphabetic cipher.**
 2. **Polyalphabetic cipher.**

MONOALPHABETIC SUBSTITUTION CIPHERS

Monoalphabetic Substitution Cipher

- **Monoalphabetic Cipher:**

- It **replaces one letter** in the plaintext by another letter.
- The replacement is a **ONE-TO-ONE** relation. That is, the same letter is replaced by the same other letter (**unique**) regardless of its position in the plaintext.
- Three major monoalphabetic cipher techniques:
 1. **Shift (Additive)** Cipher
 2. **Multiplicative** Cipher
 3. **Affine** Cipher

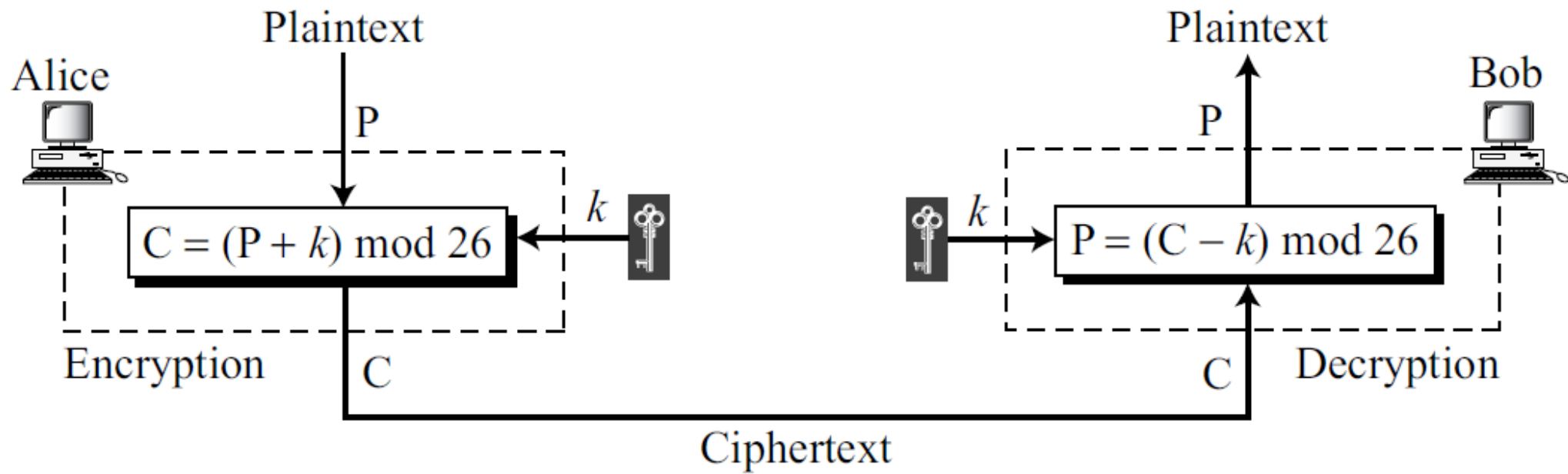
Monoalphabetic Substitution – Shift Cipher

- A technique of **shifting a key, k character** by certain position (either up or down).
- Since it involves the shift of alphabets (A – Z), there are 26 alphabets available. We can view it as a group of $\mathbb{Z}_{26}$, in which we **assign each letter to an integer**.
- It is also known as **additive** cipher.

Plaintext →	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Ciphertext →	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Value →	00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

Monoalphabetic Substitution – Shift Cipher

- This forms an **additive cipher model** with a modular of 26, or (mod 26).



Monoalphabetic Substitution – Shift Cipher

- One of the very famous shift cipher is the **Caesar cipher** due to Julius Caesar.
- This cipher specifically replaces a letter with **3 shifts down** in the alphabets, that is the key $k = 3$.
- **Examples:**
 1. $a \rightarrow D$
 2. $c \rightarrow F$
 3. $x \rightarrow A$ (if the list of alphabet reaches the end, it will cycle again from the beginning)
- These are exactly the additive ciphers with **equation** $C \equiv P + 3 \pmod{26}$. Try it yourself and observe how it works.

Monoalphabetic Substitution – Shift Cipher

➤ In general, one can choose any integer in the range of $\{0,1,2, \dots, 25\}$ as the key k in shift cipher.

➤ **Examples:**

1. If $k = 13$, then the letter $f = 05$ is encrypted into $05 + 13 \equiv 18 \pmod{26} = S$.
2. If $k = 25$, then the letter $f = 05$ is encrypted into $05 + 25 \equiv 30 \pmod{26} \equiv 04 = E$.
3. If $k = 10$, then the letter $x = 23$ is encrypted into $23 + 10 \equiv 33 \pmod{26} \equiv 07 = H$.

❖ You can try yourself using more examples. But do you notice **any interesting point**?

Monoalphabetic Substitution – Shift Cipher

- **Example 1:** Encrypt the message '**this is dr tea**' using a shift cipher with key $k = 9$.
- **Solution:** The encryption equation is given by $C \equiv P + 9 \pmod{26}$

P_i		t	h	i	s	i	s	d	r	t	e	a
		19	07	08	18	08	18	03	17	19	04	00
k		9	9	9	9	9	9	9	9	9	9	9
		28	16	17	27	17	27	12	26	28	13	09
(mod 26)		02	16	17	01	17	01	12	00	02	13	09
C_i		C	Q	R	B	R	B	M	A	C	N	J

➤ The ciphertext produced is: **CQRB RBMACNJ**.

Cryptanalysis on Shift Cipher

- Shift ciphers in particular are **vulnerable** to ciphertext-only attack using **exhausted key searches / brute-force attacks**.
- **Reason:** The **key domain is very small**, that there are only 26 possible keys.
- One needs to just try all the **26 possible shifts** (specifically 25 since $k = 0$ is usually useless) to recover the plaintext message.
- **Remember:** When decrypting the ciphertext, we **shift backwardly** the letter using the pre-determined key, since from the equation: $P \equiv C - k \pmod{26}$.

Cryptanalysis on Shift Cipher

- **Example 2:** Perform brute-force attack on the ciphertext '**GCUAVQDTGCM**'

k		G	C	U	A	V	Q	D	T	G	C	M
1		f	b	t	z	u	p	c	s	f	b	l
2		e	a	s	y	t	o	b	r	e	a	k
3		d	z	r	x	s	n	a	q	d	z	j
4		c	y	q	w	r	m	z	p	c	y	i
5		b	x	p	v	q	l	y	o	b	x	h

This gives a **meaningful** plaintext.

- The recovered plaintext is: **easy to break**.

Cryptanalysis on Shift Cipher

➤ **Exercise 1:** Perform cryptanalysis on the ciphertext '**NCJAEZRCCLASJTDPLDJ**'.

k		N	C	J	A	E	Z	R	C	L	A	S	J	T	D	P	L	D	J
1		m	b	i	z	d	y	q	b	k	z	r	i	s	c	o	k	c	i
2		l	a	h	y	c	x	p	a	j	y	q	h	r	b	n	j	b	h
3		k	z	g	x	b	w	o	z	l	x	p	g	q	a	m	l	a	g
4		j	y	f	w	a	v	n	y	h	w	o	f	p	z	l	h	z	f
:		:	:	:	:	:	:	:	:	:	:	:	:	:	:	:	:	:	:
11		c	r	y	p	t	o	g	r	a	p	h	y	i	s	e	a	s	y

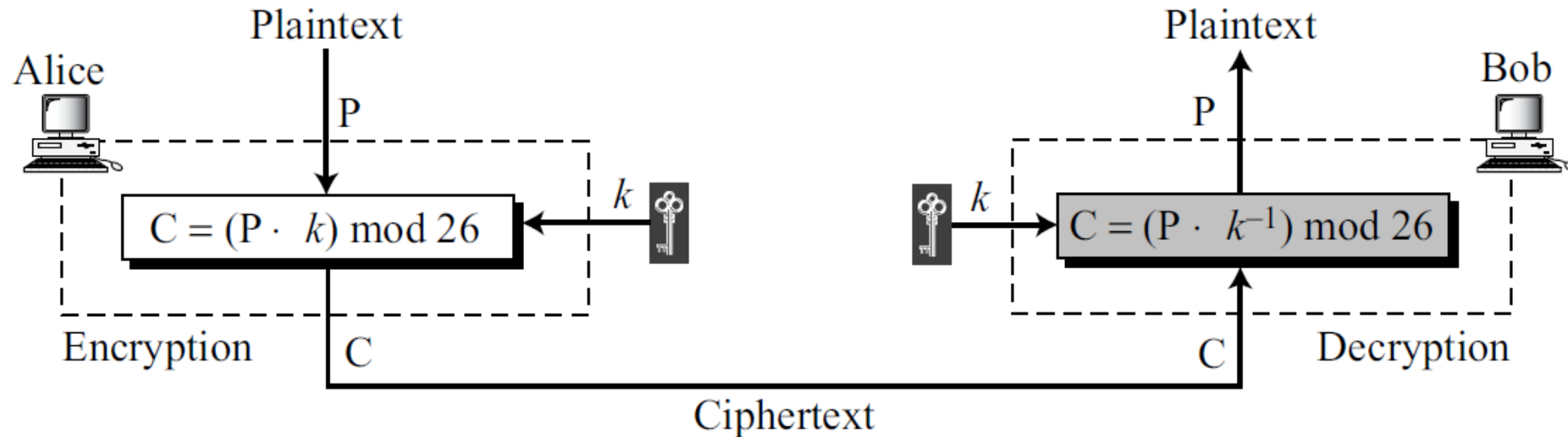
➤ The recovered plaintext is: **cryptography is easy**.

Cryptanalysis on Shift Cipher

- In short, **brute-force cryptanalysis** is easy in shift cipher due to the following **important characteristics**:
 1. The **encryption** and **decryption algorithms** are known.
 2. There are **only 25 keys** to try.
 3. The **language** of the plaintext is **known** and **easily recognizable**.

Monoalphabetic Substitution – Multiplicative Cipher

- In multiplicative cipher, the encryption algorithm specifies the **multiplication** of the plaintext by the key k , and the decryption algorithm specifies the **division** of the ciphertext by the inverse of the key, k^{-1} .



Monoalphabetic Substitution – Multiplicative Cipher

- Since the decryption requires the multiplicative inverse of the key, k^{-1} . The chosen key k **MUST BE** from the **multiplicative group** $\mathbb{Z}_{26}^*$, which guarantee the existence of multiplicative inverse of the key, k^{-1} .
- **Multiplicative group of $\mathbb{Z}_{26}^*$** : The integer $a \in \{0,1,2, \dots, 25\}$ such that $\gcd(a, 26) = 1$. Hence, there are only 12 integers $a = \{1,3,5,7,9,11,15,17,19,21,23,25\}$ that satisfy this conditions.

Monoalphabetic Substitution – Multiplicative Cipher

- **Example 3:** Encrypt the message 'wednesday' using multiplicative cipher with key $k = 7$.

➤ **Solution:** The encryption equation is given by $C \equiv P \times k \pmod{26}$.

P		w	e	d	n	e	s	d	a	y
		22	04	03	13	04	18	03	00	24
k		7	7	7	7	7	7	7	7	7
		154	28	21	91	28	126	21	00	168
mod 26		24	02	21	13	02	22	21	00	12
C		Y	C	V	N	C	W	V	A	M

➤ The ciphertext produced is: **YCVN CWVA M**.

Monoalphabetic Substitution – Multiplicative Cipher

- **Example 4:** Decrypt the ciphertext '**UAEGEAXP**' that is encrypted using multiplicative cipher with key $k = 9$.

➤ **Solution:**

1. The decryption equation is given by $P \equiv C \times 9^{-1} \pmod{26}$.
2. We need to find the multiplicative inverse of 9. By the Extended Euclidean algorithm, $9^{-1} \pmod{26} \equiv 3$.

q	n	a	r	s_1	s_2	s	t_1	t_2	t
2	26	9	8	1	0	1	0	1	-2
1	9	8	1	0	1	-1	1	-2	3
8	8	1	0						

Monoalphabetic Substitution – Multiplicative Cipher

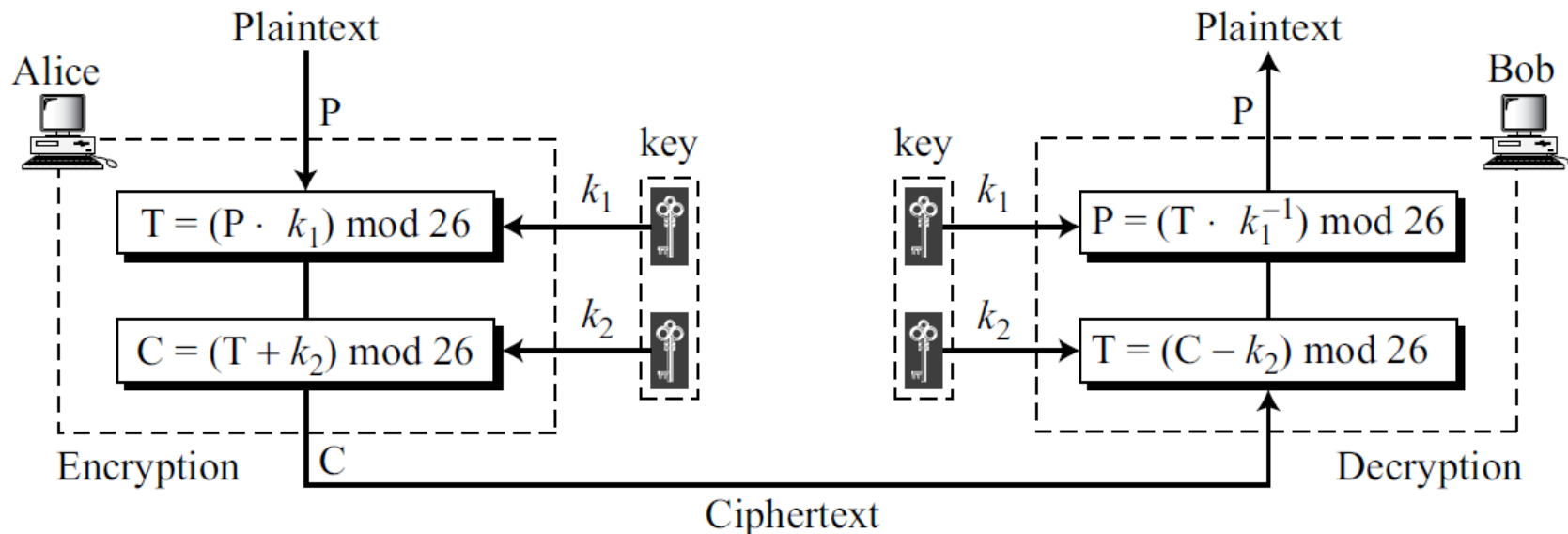
3. Hence, the decryption equation is $P \equiv C \times 3 \pmod{26}$.

C_i		U	A	E	G	E	A	X	P
		20	00	04	06	04	00	23	15
k		3	3	3	3	3	3	3	3
		60	00	12	18	12	00	69	45
mod 26		08	00	12	18	12	00	17	19
P_i		i	a	m	s	m	a	r	t

➤ The recovered plaintext is: **I am smart.**

Monoalphabetic Substitution – Affine Cipher

- The **combination** of both **additive and multiplicative** cipher techniques using a **pair of keys** (k_1, k_2).
- The plaintext is first encrypted using multiplicative cipher with key k_1 , and next encrypted further using additive cipher with key k_2 .



Monoalphabetic Substitution – Affine Cipher

- ❖ Based on the diagram, the **encryption** algorithm of the affine cipher can be simplified into the equation:

$$C \equiv (P \times k_1) + k_2 \pmod{26}$$

- ❖ Likewise, the **decryption** algorithm of the affine cipher can be simplified into the equation:

$$P \equiv (C - k_2) \times k_1^{-1} \pmod{26}$$

Monoalphabetic Substitution – Affine Cipher

- **Example 5:** Encrypt the message 'security' using affine cipher with the key pair (11,16).
- **Solution:** The encryption equation is given by $C \equiv (P \times 11) + 16 \pmod{26}$.

P_i		s	e	c	u	r	i	t	y
		18	04	02	20	17	08	19	24
k_1		11	11	11	11	11	11	11	11
k_2		16	16	16	16	16	16	16	16
		214	60	38	236	203	104	225	280
mod 26		06	08	12	02	21	00	17	20
C_i		G	I	M	C	V	A	R	U

- The ciphertext produced is: **GIMC VARU**.

Monoalphabetic Substitution – Affine Cipher

- **Exercise 2:** Perform decryption on the ciphertext '**GOJCBITONB**' that is encrypted using affine cipher with the key pair (7,25).
- **Solution:**
 1. The decryption equation is given by $P \equiv (C - 25) \times 7^{-1} \pmod{26}$.
 2. We need to find the multiplicative inverse of 7. By the Extended Euclidean algorithm, $7^{-1} \pmod{26} \equiv -11 \equiv 15$.

q	n	a	r	s_1	s_2	s	t_1	t_2	t
3	26	7	5	1	0	1	0	1	-3
1	7	5	2	0	1	-1	1	-3	4
2	5	2	1	1	-1	3	-3	4	-11
2	2	1	0						

Monoalphabetic Substitution – Affine Cipher

3. Hence, the decryption equation is $P \equiv (C - 25) \times 15 \pmod{26}$.

C_i		G	O	J	C	B	I	T	O	N	B
		06	14	09	02	01	08	19	14	13	01
k_2		25	25	25	25	25	25	25	25	25	25
k_1		15	15	15	15	15	15	15	15	15	15
		-285	-165	-240	-345	-360	-255	-90	-165	-180	-360
mod 26		01	17	20	19	04	05	14	17	02	04
P_i		b	r	u	t	e	f	o	r	c	e

➤ The recovered plaintext is: **brute force**.

Monoalphabetic Substitution – Security Issues

- All **shift**, **multiplicative** and **affine ciphers** have **small key domain**, that are a maximum of 26 possible keys only, hence they are very **vulnerable to brute-force** cryptanalysis.
- **Alternative solution for better security:** **Permute** the alphabets **randomly** instead of shifting it constantly.
- Permutation of all the 26 alphabets give you a total of about **$26! = 4.03 \times 10^{26}$ possible keys**. That's A LOT !!!
- **Note:** If you think this is secure enough, we will learn later that there is an efficient cryptanalysis on this alternative solution.

Monoalphabetic Substitution – Security Issues

- **Example 6:** Suppose now Alice and Bob agreed on the randomized alphabets between plaintext and ciphertext, as follows.

Plaintext	→	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Ciphertext	→	N	O	A	T	R	B	E	C	F	U	X	D	Q	G	Y	L	K	H	V	I	J	M	P	Z	S	W

- To encrypt a message 'hello Bob', the produced ciphertext is now '**CRDDYOYO**'.

- **Note:** There is **no specific key** to shift any of these alphabetic, it seems more secure, right?

Monoalphabetic Substitution – Frequency Analysis

- Besides brute-force attack, the most common cryptanalysis on substitution cipher is the **statistical attack** known as the **FREQUENCY ANALYSIS**.
- Assuming that common plaintext-ciphertext are written in English, and an attacker is well familiar with the nature of such a language, the **frequency of occurrence of an English letter can be predicted efficiently**.
- For example, the letter 'E' always appear with the **HIGHEST frequency** in an English text.

Monoalphabetic Substitution – Frequency Analysis

- The table below shows the **overall frequency of occurrence of letters** in an English text in general.

<i>Letter</i>	<i>Frequency</i>	<i>Letter</i>	<i>Frequency</i>	<i>Letter</i>	<i>Frequency</i>	<i>Letter</i>	<i>Frequency</i>
E	12.7	H	6.1	W	2.3	K	0.08
T	9.1	R	6.0	F	2.2	J	0.02
A	8.2	D	4.3	G	2.0	Q	0.01
O	7.5	L	4.0	Y	2.0	X	0.01
I	7.0	C	2.8	P	1.9	Z	0.01
N	6.7	U	2.8	B	1.5		
S	6.3	M	2.4	V	1.0		

Monoalphabetic Substitution – Frequency Analysis

- Besides a single letter, English words often comes in some specific letter combinations, either **two-letter (bigrams) group**, or **three-letter (trigrams) group**.
- Examples of bigrams: **TH**e, **TH**ree, **TH**ey, **TH**em, etc.
- Examples of trigrams: **THE**, **THE**me, **THE**y, **THE**ir, **THE**m, etc.
- The table below shows the groups of common bigrams and trigrams in English texts.

Digram	TH, HE, IN, ER, AN, RE, ED, ON, ES, ST, EN, AT, TO, NT, HA, ND, OU, EA, NG, AS, OR, TI, IS, ET, IT, AR, TE, SE, HI, OF
Trigram	THE, ING, AND, HER, ERE, ENT, THA, NTH, WAS, ETH, FOR, DTH

Monoalphabetic Substitution – Frequency Analysis

- The technique of frequency analysis:
 1. **Count** the relative frequencies of letters, and the resulting pattern in the ciphertext.
 2. **Compare** the relative frequencies to the standard frequency distribution for English.
 3. **Search** for the frequency of bigrams and trigrams.
 4. **Combine** the results to identify the meaningful words.
 5. **Repeat** the analysis of frequencies with **trial and error** to find a possible solution for other letters.

Monoalphabetic Substitution – Frequency Analysis

- **Why** is frequency analysis **effective** on monoalphabetic substitution cipher?
 - **Reason:** Monoalphabetic substitution **does not change the relative frequency of the letters**, be it in plaintext or ciphertext.
- This elegant method is introduced by Al-Kindi, an Arabic scientist in 9th century.
 - Mentioned in his book entitled '**Manuscript on Deciphering Cryptographic Messages**' that gave rise to the birth of cryptanalysis.

Monoalphabetic Substitution – Frequency Analysis

- **Exercise 1:** Try to use frequency analysis to break this ciphertext (It may take some time at the beginning).

OTWEWNGWCBPQABIZVQAPMLJGZWTTQVOBQUMAPMIDGZCAB
EQVBMZLZIXMLAXZQVOQVLMMXAVWEIVLLIZSNZWAB
JQZLWNLMTQOPBVIUMLGWCBPAEQNBTGTMNBBPMMVAB
ITIAKWCTLVBBQUMQBEPQTMQBEIAQVUGBZCAB

POLYALPHABETIC SUBSTITUTION CIPHERS

Polyalphabetic Substitution Cipher

- Each occurrence of **a letter** may have a **different substitute**.
- The replacement is a **ONE-TO-MANY** relation. That is, the same letter in a plaintext may be replaced by different letters in the ciphertext, **depending of its position in the plaintext and the key used**.
- In other words, the key used here is **a stream of subkeys** $(k_1, k_2, \dots, k_l)$, rather than just a single-key. Each subkey encrypts a particular letter at that position, and the **keystream is recycled** for a longer plaintext message.
- **Advantage** of polyalphabetic substitution:
 - It **hides** the letter frequency of the underlying message, hence **survive against** the **single-letter frequency analysis**.

Polyalphabetic Substitution Cipher

- Some common polyalphabetic cipher techniques:

1. **Autokey** Cipher
2. **Playfair** Cipher
3. **Vigenère** Cipher
4. **Rotor** Cipher and the **Enigma Machine**
5. **Vernam** Cipher and the **One-Time Pad**
6. **Hill** Cipher (**Optional**)

Polyalphabetic Substitution – Autokey Cipher

- A simple polyalphabetic substitution cipher.
- The key is a **stream of subkeys** used to encrypt the corresponding character in the plaintext:
 1. The **first subkey** is a **pre-determined secret value**.
 2. The **subsequent subkeys** are the **characters from the plaintext (in order)**.
- **Autokey:** The subkeys are **automatically created** from the plaintext characters during the encryption process.

$$P = P_1P_2P_3 \dots$$

$$C = C_1C_2C_3\dots$$

$$k = (k_1, P_1, P_2, \dots)$$

$$\text{Encryption: } C_i = (P_i + k_i) \bmod 26$$

$$\text{Decryption: } P_i = (C_i - k_i) \bmod 26$$

Polyalphabetic Substitution – Autokey Cipher

- **Example 7:** Using the initial subkey $k_1 = 13$, encrypt the message 'Good morning' using autokey cipher.

P_i		g	o	o	d	m	o	r	n	i	n	g
		06	14	14	03	12	14	17	13	08	13	06
k_i		13	06	14	14	03	12	14	17	13	08	13
		19	20	28	17	15	26	31	30	21	21	19
mod 26		19	20	02	17	15	00	05	04	21	21	19
C_i		T	U	C	R	P	A	F	E	V	V	T

- The ciphertext produced is: **TUCR PAFE VVT**.

Polyalphabetic Substitution – Autokey Cipher

- **Observations** from **Example 7**:
 - The character 'o' is encrypted into **two different ciphertext** characters of 'U' and 'C'.
 - The ciphertext character 'V' is an output of **two different plaintext** characters 'i' and 'n'.
 - The same plaintext character 'g' is encrypted into the same ciphertext character 'T' due to coincident of the subkey streams and its position during encryption.

Security on Autokey Cipher

- **Advantage:**
 - It **hides the single-letter frequency analysis**.
- **Disadvantage:**
 - It is still vulnerable to brute-force attack since it is an **additive cipher**.
 - An attacker may just need to **keep trying all the 25 possible pre-determined values** to recover the first character, and then use it to analyse the subsequent ciphertext (since the **subsequent subkeys are the plaintext characters**).
- **Remedy:** Use **larger key domain**.
 - Instead of a single pre-determined key, we may use **multiple pre-determined key streams**.

Polyalphabetic Substitution – Playfair Cipher

- A polyalphabetic cipher that was invented in 1854 by **Charles Wheatstone**, but named after **Baron Playfair** for promoting its use.
- The **secret key** is made of 25 alphabet letters arranged in a 5×5 matrix (Letter 'I' and 'J' are paired in the same entry)
- **Idea:**
 - The plaintext is separated into **bigrams** (2-letter pairs) as a unit, and encrypted into a ciphertext bigrams.
 - If the pair contains the same character, a **bogus letter** (usually 'x') is inserted to separate them.
 - A **keyword** (minus duplicates) is used to fill the matrix entries.

Polyalphabetic Substitution – Playfair Cipher

- **Procedures/rules** of Playfair cipher encryption:
 1. Plaintext is split into bigrams, **repeating letters** that are **in the same pair** and **last single letter** are **separated/padded with a bogus** (usually 'x').
 2. Two plaintext letters that fall in the **same row** of the matrix are each replaced by the letter to the **RIGHT**, with the 1st element of the row circularly following the last.
 3. Two plaintext letters that fall in the **same column** of the matrix are each replaced by the letter **BENEATH**, with the top element of the column circularly following the last.
 4. Otherwise, each plaintext letter in pair is replaced by the letter that **lies in its own row and the column** occupied by the other plaintext letter.

Polyalphabetic Substitution – Playfair Cipher

- **Example 8:** Perform encryption on the message '**parallel**' using Playfair cipher with the keyword '**monday**'.

- **Solution:**

1. The plaintext is split into bigrams: pa ra lx le lx.
2. Fill the matrix with **keyword first**, then followed by the remaining letters in **alphabetical order** (see right).
3. Encrypt according the procedures/rules given.

M	O	N	D	A
Y	B	C	E	F
G	H	I/J	K	L
P	Q	R	S	T
U	V	W	X	Z

Polyalphabetic Substitution – Playfair Cipher

4. According to the matrix, the bigram 'pa' is encrypted into 'TM'. 'p' slides to 'T' and 'a' slides to 'M' (you can view as they are **sliding to the corner** end of each row and column in the 'box').
 5. The procedures are repeated for all the bigrams.
- The produced ciphertext is: **TMTN KZKF KZ**.

Plaintext	Ciphertext
pa	TM
ra	TN
lx	KZ
le	KF
lx	KZ

Polyalphabetic Substitution – Playfair Cipher

- **Example 9:** Perform encryption on the message '**balloon**' using Playfair cipher with the keyword '**monarchy**'.

- **Solution:**

1. The plaintext is split into bigrams: ba lx lo on.
2. Fill the matrix with **keyword first**, then followed by the remaining letters in **alphabetical order** (see right).
3. Encrypt according the procedures/rules given.

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

Polyalphabetic Substitution – Playfair Cipher

4. According to the matrix, the bigram 'ba' is encrypted into 'IB' or 'JB' (whichever you prefer). 'b' drops to 'I/J' and 'a' drops to 'B'.
5. The procedures are repeated for all the bigrams.

- The produced ciphertext is: **IB(JB)SU PMNA**.

Plaintext	Ciphertext
ba	IB/JB
lx	SU
lo	PM
on	NA

Polyalphabetic Substitution – Playfair Cipher

- **Exercise 4:** Perform encryption on the message 'The class is cancelled today' using Playfair cipher with the keyword 'cryptography'.

- **Solution:**

1. The plaintext is split into bigrams:
th ec la sx si sc an ce lx le dt od ay.

2. The produced ciphertext is:

PBDR NOQZ QKLT FWRD QUMD KCDL FA.

C	R	Y	P	T
O	G	A	H	B
D	E	F	I/J	K
L	M	N	Q	S
U	V	W	X	Z

Security on Playfair Cipher

- This cipher was once used as standard field system by the British army in World War I and U.S. army in World War II.
- **Advantages:**
 - **Security much improved** over monoalphabetic cipher.
 - **Brute-force attack is very difficult** since it has $25!$ size of key domain.
- **Disadvantage:**
 - **Frequencies of bigrams are preserved.** A cryptanalyst can use a **ciphertext-only attack** based on the bigram frequency test to find the key.

Polyalphabetic Substitution – Vigenère Cipher

- The best known, and one of the **simplest polyalphabetic substitution** cipher named after **Blaise de Vigenère**, a French Mathematician.
- **Idea:**
 - The **key stream is a repetition of an initial secret key stream** of length m , where $1 \leq m \leq 26$.
 - The key stream **does not** depend on the plaintext characters, but its **position**. That is, one can create the key stream **without knowing** the content of the plaintext.

$$P = P_1P_2P_3 \dots$$

$$C = C_1C_2C_3 \dots$$

$$K = [(k_1, k_2, \dots, k_m), (k_1, k_2, \dots, k_m), \dots]$$

$$\text{Encryption: } C_i = P_i + k_i$$

$$\text{Decryption: } P_i = C_i - k_i$$

Polyalphabetic Substitution – Vigenère Cipher

- **Example 10:** Perform encryption on the message '**it is raining now**' using Vigenère cipher with the keyword '**cloud**'.

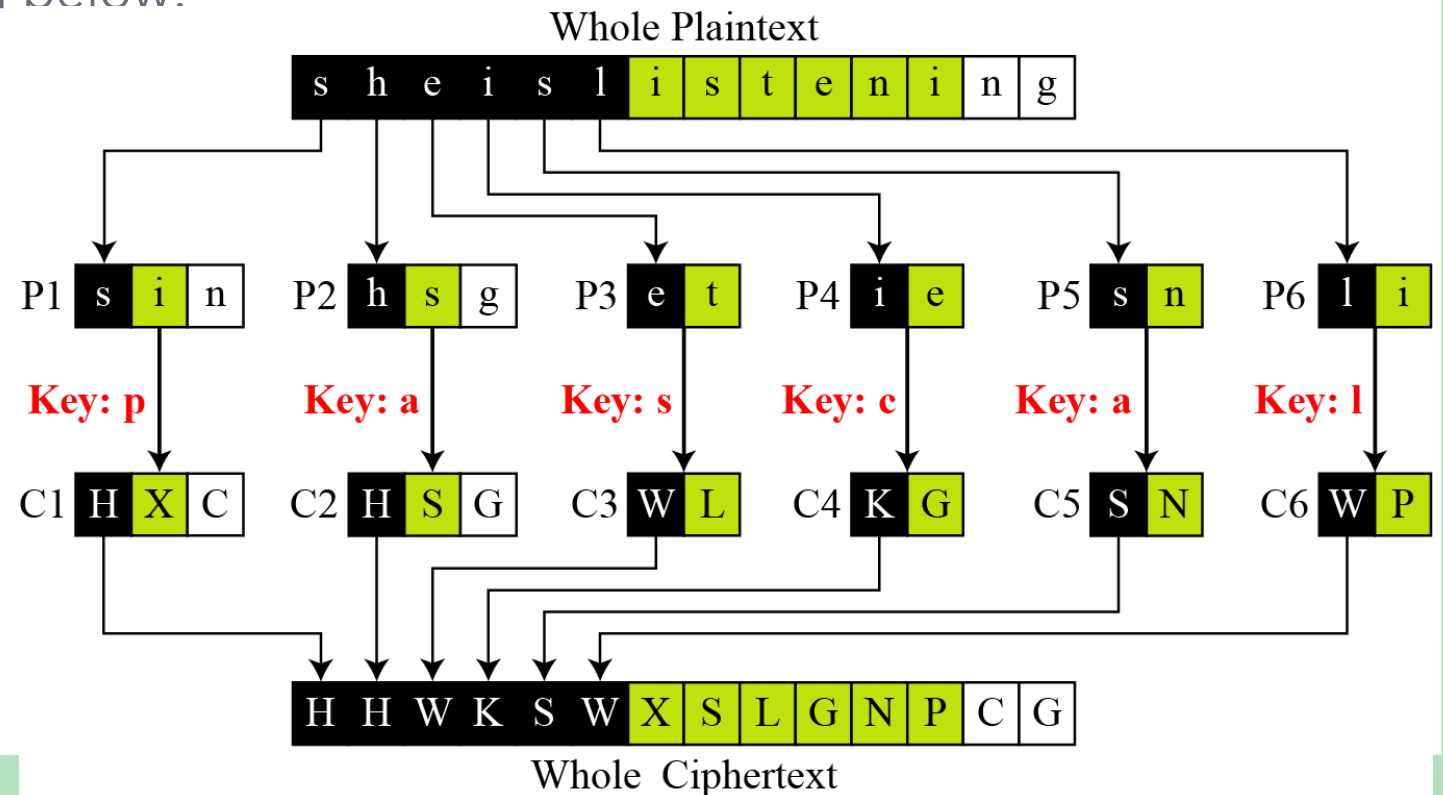
P_i		i	t	i	s	r	a	i	n	i	n	g	n	o	w
		08	19	08	18	17	00	08	13	08	13	06	13	14	22
Keyword		c	l	o	u	d	c	l	o	u	d	c	l	o	u
k_i		02	11	14	20	03	02	11	14	20	03	02	11	14	20
		10	30	22	38	20	02	19	27	28	16	08	24	28	42
mod 26		10	04	22	12	20	02	19	01	02	16	08	24	02	16
C_i		K	E	W	M	U	C	T	B	C	Q	I	Y	C	Q

- The produced ciphertext is : **KEWMUCTBCQIYCO**.

Polyalphabetic Substitution – Vigenère Cipher

- In another perspective, we can view the Vigenère cipher as a combination of m additive ciphers, as illustrated below:

- (Right) Encryption of the message '**she is listening**' using the keyword '**pascal**' that produces ciphertext '**HHWK SWXS LGNP CG**'.



Polyalphabetic Substitution – Vigenère Cipher

- Alternatively, we can use a **Vigenère tableau** (in next slide) to find the output of the encryption.
 - ✓ The **top row** (small letters) represents the **plaintext** characters.
 - ✓ The **leftmost column** (capital letters) represents the **keyword**.
 - ✓ The **remaining entries** in the table are the **ciphertext** characters.
- **Example:** The message '**we are here to sleep**' is encrypted using the keyword '**deceptive**'. Using the Vigenère tableau:

P_i		w	e	a	r	e	h	e	r	e	t	o	s	i	e	e	p
k_i		d	e	c	e	p	t	i	v	e	d	e	c	e	p	t	i
C_i		Z	I	C	V	T	A	M	M	I	W	S	U	P	T	X	X

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	v	v	w	x	y	z
<i>A</i>	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
<i>B</i>	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
<i>C</i>	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
<i>D</i>	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
<i>E</i>	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
<i>F</i>	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
<i>G</i>	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
<i>H</i>	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
<i>I</i>	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
<i>J</i>	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
<i>K</i>	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
<i>L</i>	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
<i>M</i>	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
<i>N</i>	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
<i>O</i>	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
<i>P</i>	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
<i>Q</i>	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
<i>R</i>	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
<i>S</i>	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
<i>T</i>	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
<i>U</i>	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
<i>V</i>	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
<i>W</i>	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
<i>X</i>	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
<i>Y</i>	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
<i>Z</i>	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Polyalphabetic Substitution – Vigenère Cipher

- **Exercise 4:** Perform encryption on the message 'The class is cancelled today' using Vigenère cipher with the keyword 'cryptography'.
- **Solution:**

t	h	e	c	l	a	s	s	i	s	c	a	n	c	e	l	l	e	d	t	o	d	a	y
C	R	Y	P	T	O	G	R	A	P	H	Y	C	R	Y	P	T	O	G	R	A	P	H	Y
V	Y	C	R	E	O	Y	J	I	H	J	Y	P	T	C	A	E	S	J	K	O	S	H	W

- The ciphertext produced is: **VYCR EOYJ IHJY PTCA ESJK OSHW**.

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	v	v	w	x	y	z
<i>A</i>	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
<i>B</i>	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
<i>C</i>	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
<i>D</i>	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
<i>E</i>	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
<i>F</i>	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
<i>G</i>	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
<i>H</i>	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
<i>I</i>	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
<i>J</i>	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
<i>K</i>	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
<i>L</i>	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
<i>M</i>	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
<i>N</i>	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
<i>O</i>	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
<i>P</i>	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
<i>Q</i>	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
<i>R</i>	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
<i>S</i>	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
<i>T</i>	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
<i>U</i>	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
<i>V</i>	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
<i>W</i>	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
<i>X</i>	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
<i>Y</i>	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
<i>Z</i>	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Security on Vigenère Cipher

- **Advantage:**

- Vigenère cipher **does not preserve** the frequency of characters, hence the letter frequencies are hidden.

- **Disadvantage:**

- The knowledge of the plaintext structure is **not lost completely**.
- Cryptanalysis is still possible via:
 1. Finding the **key length**. For example, the **Kasiski test**.
 2. Finding the **key**.

Security on Vigenère Cipher

- **Cryptanalysis** on Vigenère cipher (common practice):
 1. Determine the **length of the keyword**.
 - If two identical sequences of plaintext letters occur at a distance that is an integer multiple of the keyword length, they will generate identical ciphertext sequences.
 - By looking for common factors in the **displacements** of the various repeated sequences in the ciphertext, the analyst guess the length of keyword.
 2. If the keyword length is m , then the cipher consists of **m monoalphabetic ciphers**.
 3. Use the **frequency analysis** of the plaintext language to attack each of the monoalphabetic ciphers separately.

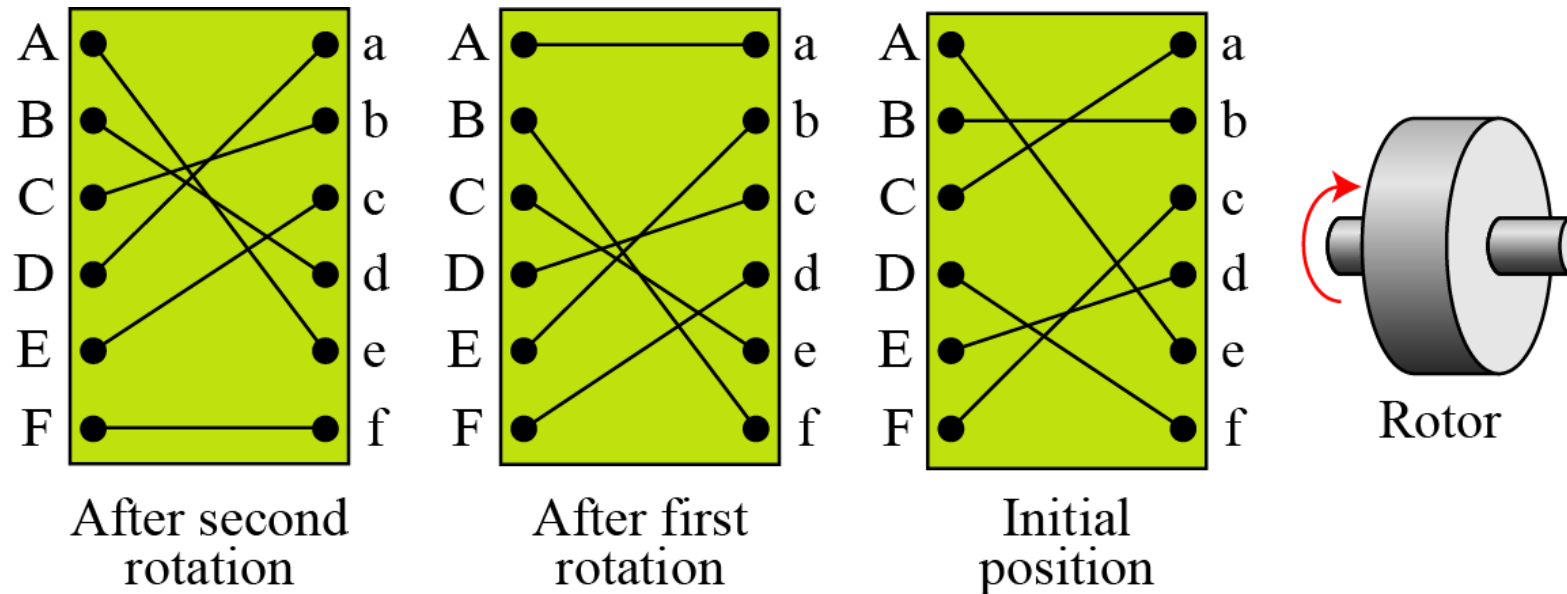
Polyalphabetic Substitution – Rotor Cipher

- **Idea:**

- It uses the monoalphabetic substitution technique, but changes the mapping between the plaintext characters and the ciphertext characters for each plaintext character.
- In other word, after encrypting one plaintext character, the rotor will rotate (according to the pre-defined mapping) and the **next encryption for the same plaintext character may NOT necessarily be the same.**

Polyalphabetic Substitution – Rotor Cipher

- The example of **rotor cipher** is shown below. If one encrypts the message '**deaf**', then the ciphertext produced will be '**ECDC**'. (Do you know why '**f**' is encrypted to '**C**'?)

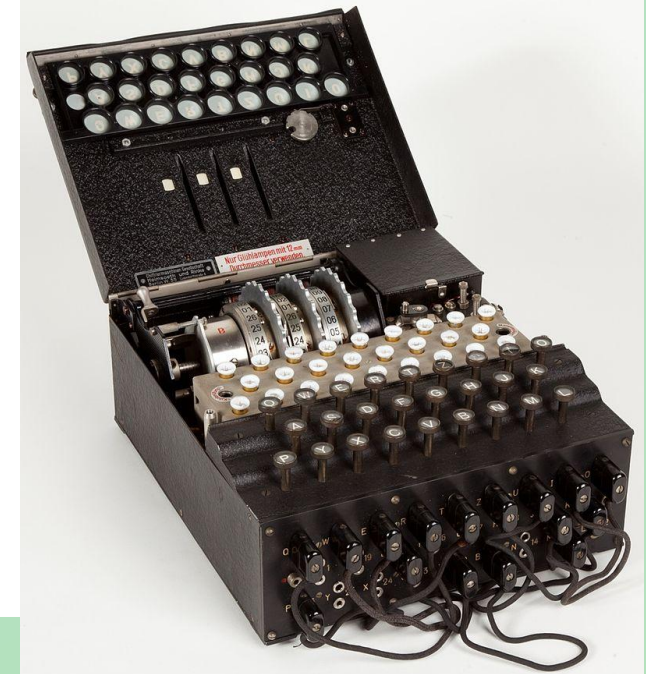


Security on Rotor Cipher

- The demonstrated example is just taking **6 letters in the rotor**, and **permute (map)** them **randomly**., there will be **$6! = 720$ different possible mappings**. Imagine if this is done on 26 letters.....
- **Advantage:**
 - It **resistant** to **brute-force attack**.
 - It is also **much more resistant** to **statistical attack**, as it **does NOT preserve** letter frequency

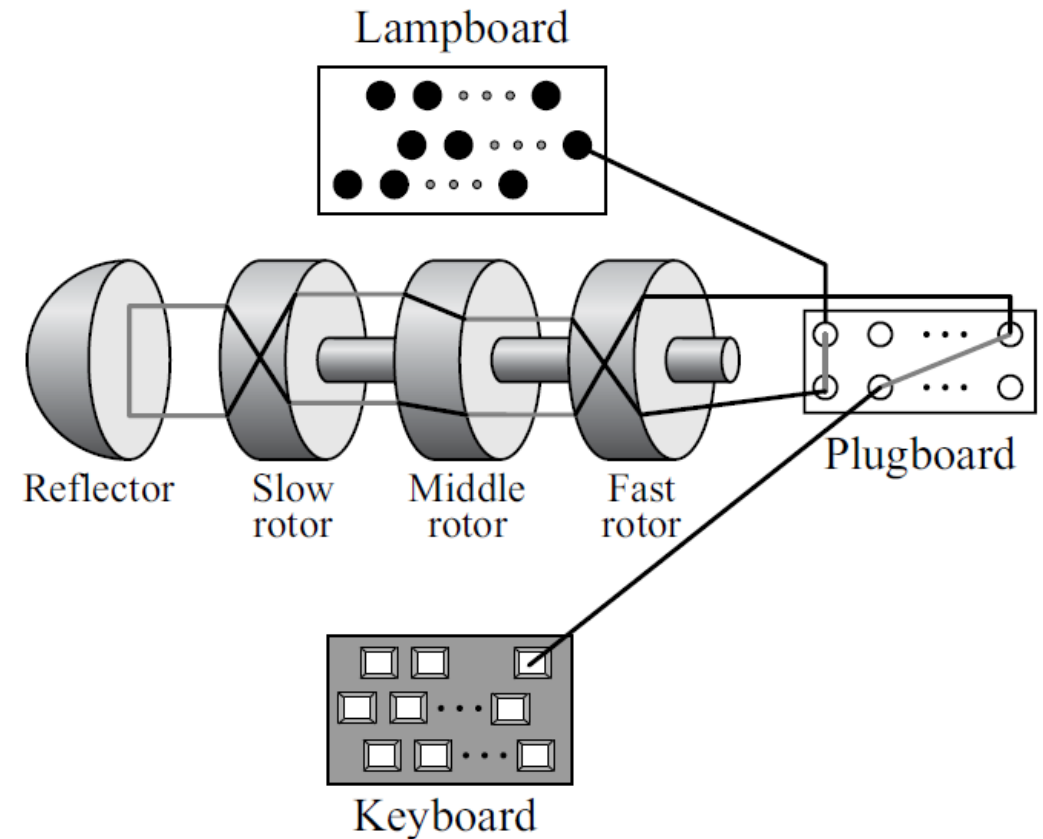
Advancement on Rotor Cipher – the Enigma Machine

- The **Enigma Machine** was designed based on **the idea of rotor cipher**.
- It was originally invented by Sherbius, but later modified by German army and **used extensively during the World War II**.
- The Enigma Machine defeated all the existing cryptography techniques at that time, **thought to be very secure**.
- This machine was broken by **Alan Turing**, which believed to **shorten the WWII by at least 2 years**.



Advancement on Rotor Cipher – the Enigma Machine

- The right shows the schematic of the Enigma machine.
- The complete and detailed procedures can be referred to the main reference book by Forouzan [1], page 79 – 80.



Polyalphabetic Substitution – Vernam Cipher

- The Vernam cipher was introduced by **Gilbert Vernam** in 1918.
- **Idea:**
 - The plaintext is encrypted using a keystream that is having the **SAME LENGTH** of the plaintext.
 - Instead of using binary operations as in other substitution cipher, Vernam cipher operates on **binary data (with bits of 0 and 1) using exclusive-OR function.**
- Hence,

$$\text{plaintext} \oplus \text{key} = \text{ciphertext}$$

$$\text{ciphertext} \oplus \text{key} = \text{plaintext}$$

Advancement on Vernam Cipher – the One-Time Pad

- Advanced from the idea of Vernam cipher, the **One-Time Pad** is the **ONLY** cipher that is **UNBREAKABLE** !!! A **perfect cipher with perfect secrecy** (wow). 🤖
- This cipher is encrypted using a **completely random chosen key** that has the **SAME length** as the plaintext.

**Why not everyone use it,
since it is so perfect, right?**

Advancement on Vernam Cipher – the One-Time Pad

- **Practical issue of the One-Time Pad:**
- Almost impossible to implement it commercially, since each communication requires a newly generated secret key that is as long as plaintext text.
 1. **Producing large quantities** of random keys becomes a challenging task.
 2. **Distributing and protecting** these keys becomes a problem.
- Still, it is useful for small-scale implementation such as sending small secret message between you and your friends.

Polyalphabetic Substitution – Hill Cipher (Optional)

- ❖ A polyalphabetic cipher invented by **Lester S. Hill** in 1929 that is based on **Linear Algebra (matrices)**.
- ❖ **Idea:**
 - Unlike all the previously discussed cipher techniques which encrypt message as **plaintext stream**, Hill cipher is performed via **dividing plaintext into equal-size blocks**. Hence, it is categorized as **block ciphers**.
 - The **key is a square matrix of dimension $m \times m$** . Here, the m represents the block size.

Polyalphabetic Substitution – Hill Cipher (Optional)

- In general, let $\mathbf{K}$ be the $m \times m$ key matrix, and $\mathbf{P}$ be the column vector (a $m \times 1$ matrix), then:

$$\mathbf{C} = \mathbf{KP} \pmod{26}$$

- Recall that in **modular multiplication operation**, the **multiplicative inverse must exists** for reverse operation to be valid. Hence, in this **modular matrix multiplication**, the **multiplicative inverse** corresponds to its **DETERMINANT**.

Polyalphabetic Substitution – Hill Cipher (Optional)

- **Example 11**: Encrypt the message 'pay' using Hill cipher with the following key.

$$K = \begin{pmatrix} 17 & 17 & 5 \\ 21 & 18 & 21 \\ 2 & 2 & 19 \end{pmatrix}$$

- **Solution**: Assign the message their corresponding integer in $\mathbb{Z}_{26}$ to form a message vector.

$$P = \begin{pmatrix} 15 \\ 0 \\ 24 \end{pmatrix}$$

Polyalphabetic Substitution – Hill Cipher (Optional)

- Then,

$$\mathbf{C} = \mathbf{KP} \pmod{26} = \begin{pmatrix} 17 & 17 & 5 \\ 21 & 18 & 21 \\ 2 & 2 & 19 \end{pmatrix} \begin{pmatrix} 15 \\ 0 \\ 24 \end{pmatrix} = \begin{pmatrix} 375 \\ 819 \\ 486 \end{pmatrix} = \begin{pmatrix} 11 \\ 13 \\ 18 \end{pmatrix} \pmod{26}$$

- The produced ciphertext is: $\begin{pmatrix} 11 \\ 13 \\ 18 \end{pmatrix} = \begin{pmatrix} L \\ N \\ S \end{pmatrix}$.

Polyalphabetic Substitution – Hill Cipher (Optional)

- To perform decryption, one needs to find the **inverse key matrix K^{-1}** , and compute

$$\mathbf{P} = \mathbf{K}^{-1}\mathbf{C} \pmod{26}$$

- With $\mathbf{K}^{-1}\mathbf{K} = \mathbf{I} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$.
- **Note:** We may not have enough time to simulate the Hill cipher in the lecture class, but we will do it during the **tutorial sessions**, as it requires some time to compute the matrix and multiplicative inverses (if you are not familiar or forgotten about matrices).

Security on Hill Cipher (Optional)

❖ Advantage:

- The Hill cipher **hides the single-letter frequency information**, therefore survive the frequency analysis.
- The larger dimension of key matrix provide higher security level.

❖ Disadvantage:

- It is **computational more costly** since larger dimension of key matrices requires more computational power.

TRANSPOSITION CIPHERS

Transposition Ciphers - Overview

- Transposition ciphers **does not** substitute one symbol for another, instead it **changes the location (or position)** of the symbols.
- In other words, transposition ciphers **REORDER (or permute)** the symbols **without changing** its actual (original) symbols.
- Transposition ciphers can be further divided into **two categories**:
 1. **Keyless Transposition cipher.**
 2. **Keyed Transposition cipher.**

KEYLESS TRANSPOSITION CIPHERS

Keyless Transposition - Overview

- This technique **does not** required any **key** in permuting the plaintext characters.
- There are generally **two approaches** in the keyless transposition cipher:
 1. The **plaintext** characters are arranged **column-by-column**, and the **ciphertext** is written **row-by-row**.
 2. The **plaintext** characters are arranged **row-by-row**, and the **ciphertext** is written **column-by-column**.

Keyless Transposition – Rail Fence Cipher

- The **Rail Fence cipher** is the example of 1st approach in keyless transposition cipher.
- It arranges the **plaintext into two lines as a zigzag pattern** (column-by-column), and next the corresponding **ciphertext is read and written row-by-row**.

- **Example 12**: Encrypt the message 'meet you tonight' using **rail fence** cipher.

m		e		y		u		o		i		h	
	e		t		o		t		n		g		t

- **Solution**:
 - The produced ciphertext is: **MEYU OIHE TOTN GT**.

Keyless Transposition – Rail Fence Cipher

- Plaintext: **the weather today is so hot and dry**
- Depth (Rows): 2
- Ciphertext: **TEETE TDYSO OADRH WAHRO AISHT NDY**

t		e		e		t		e		t		d		y		s		o		o		a		d		r	
	h		w		a		h		r		o		a		i		s		h		t		n		d		y

Keyless Transposition – Rail Fence Cipher

- Plaintext: **the weather today is so hot and dry**
- Depth (Rows): 3
- Ciphertext: **TEEDS ODHWA HROAI SHTND YETTY OAR**

t				e				e				d				s				o				d			
	h		w		a		h		r		o		a		i		s		h		t		n		d		y
		e				t				t				y				o				a				r	

Keyless Transposition – Rail Fence Cipher

- Plaintext: **the weather today is so hot and dry**
- Depth (Rows): 4
- Ciphertext: **TTDOD HAHOA SHNDE EETYS OARWR ITY**

t						t						d						o						d			
	h				a		h				o		a				s		h				n		d		
		e		e				e		t				y		s				o		a				r	
			w						r						i						t						y

Keyless Transposition Cipher

- Consider the same **Example 12**, we now encrypt this message using the 2nd approach in keyless transposition cipher (see right).
- Here, the plaintext message '**meet you tonight**' is arranged row-by-row first, next the ciphertext is written column-by-column.
- The produced ciphertext is: **MYOH EONT EUIT TG**.
- **Note:** The number of columns is upon personal preferences. For a longer plaintext message, one may consider adding more columns.

m	e	e	t
y	o	u	t
o	n	i	g
h	t		

Keyless Transposition Cipher

- Plaintext: **the weather today is so hot and dry**
- Columns: 5
- Ciphertext:
TATIOD HTOSTR EHDSAY WEAONX ERYHDZ

t	h	e	w	e
a	t	h	e	r
t	o	d	a	y
i	s	s	o	h
o	t	a	n	d
d	r	y	x	z

Keyless Transposition Cipher

- Plaintext: **the weather today is so hot and dry**
- Columns: 6
- Ciphertext:
TTDOD HHAHD EEYOR WRITY ETSAX AOSNZ

t	h	e	w	e	a
t	h	e	r	t	o
d	a	y	i	s	s
o	h	o	t	a	n
d	d	r	y	x	z

Keyless Transposition Cipher

- Plaintext: **the weather today is so hot and dry**
- Columns: 7
- Ciphertext:
THYT HEIA ERSN WTSD WOOD ADHR TAOY

t	h	e	w	e	a	t
h	e	r	t	o	d	a
y	i	s	s	o	h	o
t	a	n	d	d	r	y

Security on Keyless Transposition Cipher

- The **permutation** of the keyless transposition cipher is done on the whole plaintext message to create whole ciphertext.
- As no key is involved in the keyless transposition cipher, it is very vulnerable to many cryptanalytic attacks, especially **ciphertext-only attack**.
- One may even to **able to guess** whether keyless transposition cipher has been used based on the **ciphertext itself** only. (Logically do you know why?)

KEYED TRANSPOSITION CIPHERS

Keyed Transposition Ciphers

- In **keyed** transposition cipher, the plaintext is split into blocks of pre-determined fixed size.
- The **permutation** is then performed upon these **blocks of plaintext** using a **key**. This key determines the permutations of the plaintext characters in each block.

Keyed Transposition Ciphers

- **Example 13:** Encrypt the message 'meet you tonight at your house' using transposition cipher with key (5,1,2,4,3).

- **Solution:**

1. Since the key given consist of 5 numbers, the message is parted into 5-letter blocks.

m	e	e	t	y
---	---	---	---	---

o	u	t	o	n
---	---	---	---	---

i	g	h	t	a
---	---	---	---	---

t	y	o	u	r
---	---	---	---	---

h	o	u	s	e
---	---	---	---	---

2. Next, permute the message in each block using the key

Y	M	E	T	E
---	---	---	---	---

N	O	U	O	T
---	---	---	---	---

A	I	G	T	H
---	---	---	---	---

R	T	Y	U	O
---	---	---	---	---

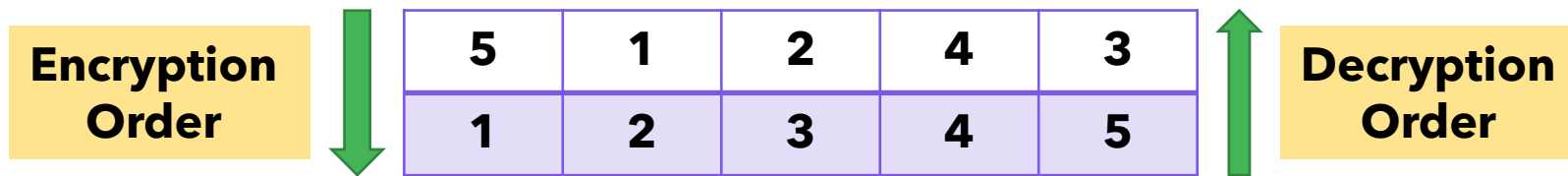
E	H	O	S	U
---	---	---	---	---

Keyed Transposition Ciphers

3. Now, the ciphertext can be written and sent as:

YMETE NOUOT AIGTH ATMUO EHOSU

- When receiver receives the ciphertext, he will first find the inverse of the permutation key (5,1,2,4,3), and next permute these ciphertext letter in the similar way to recover the message



- In this case, the **inverse permutation key is (2, 3, 5, 4, 1)**.

Keyed Transposition Ciphers

- **Example 14:** Encrypt the message 'meet you tonight at your house' using transposition cipher with key (6,2,5,1,3,4).

- **Solution:**

1. Since the key given consist of 6 numbers, the message is parted into 6-letter blocks.

m	e	e	t	y	o	u	t	o	n	i	g	h	t	a	t	y	o	u	r	h	o	u	s	e	x	z	x	z	x
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

2. Next, permute the message in each block using the key

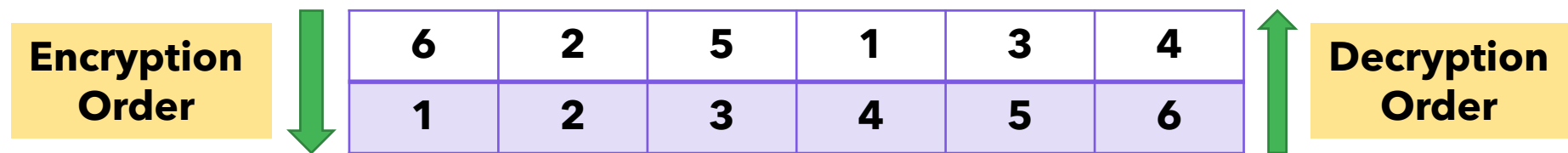
O	E	Y	M	E	T	G	T	I	U	O	N	O	T	Y	H	A	T	S	R	U	U	H	O	X	X	Z	E	Z	X
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Keyed Transposition Ciphers

3. Now, the ciphertext can be written and sent as:

OEYMET GTIUON OTYHAT SRUUHO XXZEZX

- When receiver receives the ciphertext, he will first find the inverse of the permutation key (6,2,5,1,3,4), and next permute these ciphertext letter in the similar way to recover the message



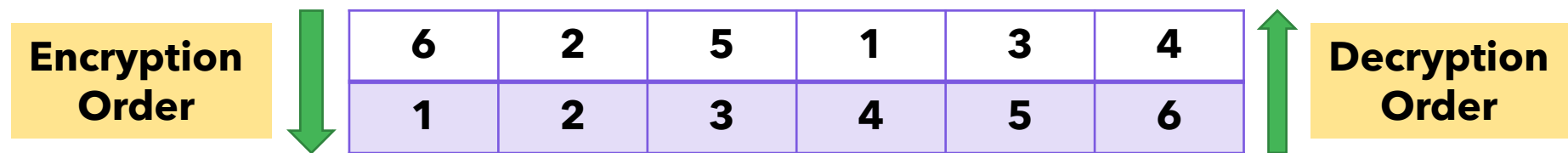
- In this case, the **inverse permutation key is**

Keyed Transposition Ciphers

3. Now, the ciphertext can be written and sent as:

OEYMET GTIUON OTYHAT SRUUHO XXZEZX

- When receiver receives the ciphertext, he will first find the inverse of the permutation key (6,2,5,1,3,4), and next permute these ciphertext letter in the similar way to recover the message



- In this case, the **inverse permutation key is (4, 2, 5, 6, 3, 1)**.

KEYLESS + KEYED TRANSPOSITION CIPHERS

(**Columnar** Transposition Ciphers)

Keyless + Keyed Transposition Cipher

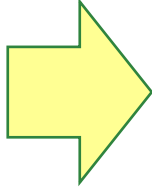
- To **increase the security of the transposition cipher**, both keyless and keyed approaches are used instead of one.
- The **encryption process** is done in **three (3) steps**:
 1. The **plaintext** is written into a table **row-by-row**. The **number of columns** is dependent on the **permutation key** used. If there is any remaining **blank entry** in the table, **pad it with some random letters** (usually 'x' or 'z').
 2. **Permutation** is done by reordering the **columns**.
 3. The **permuted ciphertext** table is read **column-by-column**.

Keyless + Keyed Transposition Cipher

- **Example 15:** Using the same plaintext '**meet you tonight at your house**' and key (5,1,2,4,3) in **Example 13**. Perform the **encryption** using the combination of keyless and keyed transposition cipher.
- **Solution:** Refer to the three steps given in the previous slides.

- The produced ciphertext is:
YNARE MOITH EUGYO TOTUS ETHOU

1	2	3	4	5
m	e	e	t	y
o	u	t	o	n
i	g	h	t	a
t	y	o	u	r
h	o	u	s	e



5	1	2	4	3
y	m	e	t	e
n	o	u	o	t
a	i	g	t	h
r	t	y	u	o
e	h	o	s	u

Keyless + Keyed Transposition Cipher

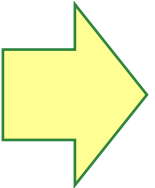
- To decrypt the ciphertext using this combined approach, we reverse the 3 steps:
 1. The **permuted ciphertext** is written into a table **column-by-column**.
 2. **Permutation** is **inverted** by reordering the **columns** using the **inverted permutation key**.
 3. The **plaintext** table is read **row-by-row**.

Security on Transposition Cipher

- Transposition cipher **DOES NOT** change the letter frequency in the ciphertext, it **only REORDERS** the letters. Hence, it is **vulnerable to frequency analysis** (statistical attack).
- Since the plaintext is written into a table, an attacker could guess the **key-length (number of columns)** used in the ciphertext, this leads to **brute-force attack**. This attack is easier for a shorter-length plaintext.
- **Pattern attack**: Some ciphertext may reveal some **repeated patterns**, resulting attack able to **guess the number of columns** used in the keyed transposition cipher.
- **Security enhancement**: **Double** transposition cipher. The same process is repeated for the 1st output ciphertext to generate 2nd ciphertext.

Keyless + Keyed Transposition Cipher

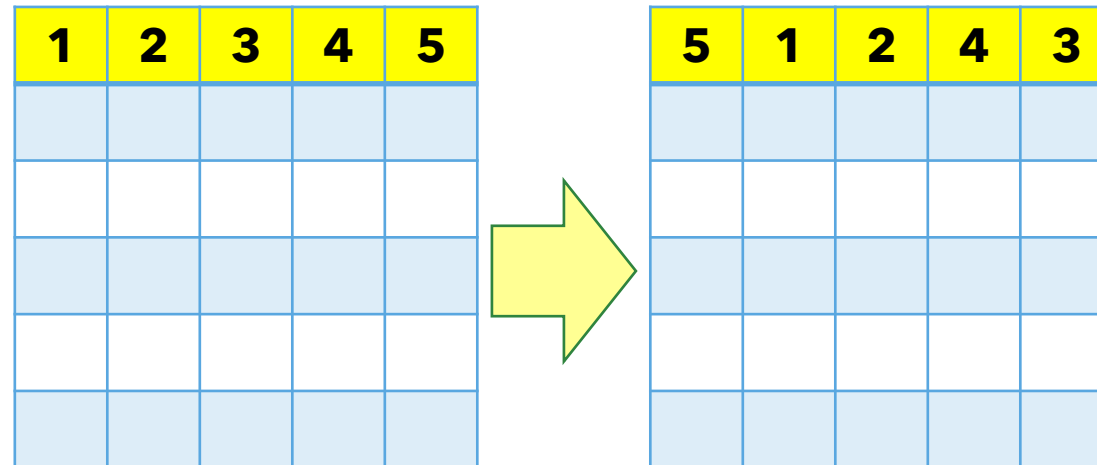
- Using the same plaintext '**meet you tonight at your house**' and key (5,1,2,4,3). The following is the output of the FIRST encryption.

1	2	3	4	5						5	1	2	4	3
m	e	e	t	y		y	m	e	t	e				
o	u	t	o	n		n	o	u	o	t				
i	g	h	t	a		a	i	g	t	h				
t	y	o	u	r		r	t	y	u	o				
h	o	u	s	e		e	h	o	s	u				

Ciphertext: **YNARE MOITH EUGYO TOTUS ETHOU**

Keyless + Keyed Transposition Cipher

- Now, we perform SECOND stage of Columnar Transposition Cipher, using the same permutation key (5,1,2,4,3)
- From 1st encryption : **YNARE MOITH EUGYO TOTUS ETHOU**

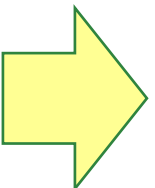


Ciphertext:

Keyless + Keyed Transposition Cipher

- Now, we perform SECOND stage of Columnar Transposition Cipher, using the same permutation key (5,1,2,4,3)

1	2	3	4	5
y	n	a	r	e
m	o	i	t	h
e	u	g	y	o
t	o	t	u	s
e	t	h	o	u



5	1	2	4	3
e	y	n	r	a
h	m	o	t	i
o	e	u	y	g
s	t	o	u	t
u	e	t	o	h

Ciphertext: **EHOSU YMETE NOUOT RTYUO AIGTH**

PRODUCT CIPHERS

Product Cipher - Overview

- Ciphers using substitutions **or** transpositions are not secure because of language characteristics.
- Hence, **product cipher** uses several ciphers in succession or combination to make the ciphertext harder to cryptanalyze.
- **Examples:**
 1. Two substitutions make a more complex substitution.
 2. Two transpositions make a more complex transposition.
 3. A substitution followed by a transposition makes a new much secure cipher.

Main References for Chapter 3

1. Forouzan, B.A. "**Introduction to Cryptography and Network Security**", Third Edition, McGraw-Hill, 2008.
2. Stallings, W. "**Cryptography and Network Security: Principles and Practices**", Fifth Edition, Pearson Prentice Hall, 2011.

~ THE END ☺ ~

Keyless Transposition – Rail Fence Cipher

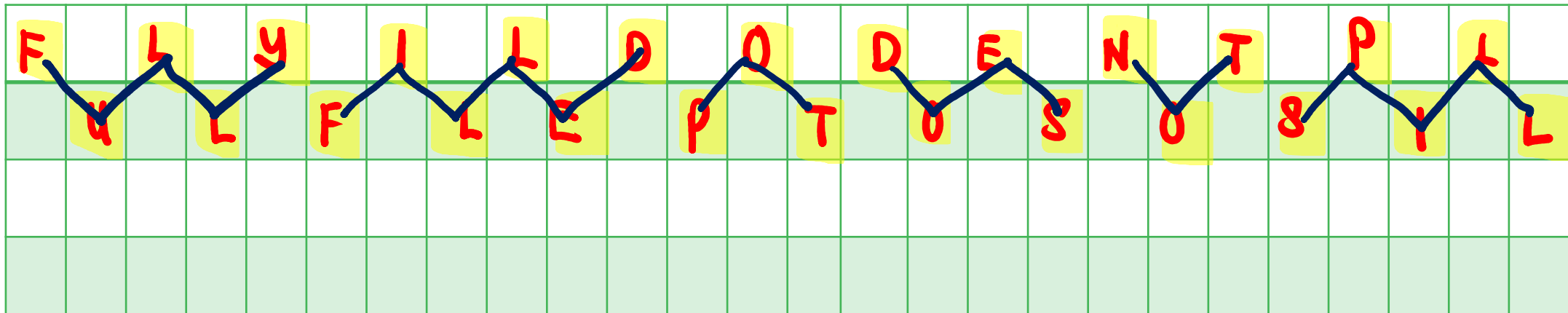
FLYILDODENTPLULFLEPTOSOSIL

Fully filled pot does not spin

Depth : 2 ←

 : 3

 : 4



Keyless Transposition – Rail Fence Cipher



FLYILDODENTPLULFLEPTOSOSIL

Depth : 3

[illegible]

Keyless Transposition – Rail Fence Cipher



FLYILDODENTPLULFLEPTOSOSIL

Depth = 4

F						L					Y						I						L	
	D				O		D				E		N				T		P				L	U
		L		F				L		E			P		T				O		S			
			O						S					I							L			

WUEA IXBA PYRS NHSR PSTS BHRT OÄFL

STTB AIUS OUHA FHIZ EIOI ITHT ARNZ

W	P	P	O	A	F	I
U	Y	S	A	I	H	T
E	R	T	F	U	I	H
A	S	S	L	S	Z	T
I	N	B	S	O	E	A
X	H	H	T	U	I	R
B	S	R	T	H	O	N
A	R	T	B	A	I	Z

1st
Dec.
→

P	O	F	A	W	P	I
Y	A	H	I	U	S	T
R	F	I	U	E	T	H
S	L	Z	S	A	S	T
N	S	E	O	I	B	A
H	T	I	U	X	H	R
S	T	O	H	B	R	N
R	B	I	A	A	T	Z

1 2 3 4 5 6 7

2 4 6 5 1 3 7

	↓	↓	↓	↓	↓	
P	A	I	S	I	H	N
O	H	U	A	B	R	R
F	I	E	S	A	S	B
A	U	T	T	H	T	I
W	S	H	N	T	O	A
P	T	S	S	I	H	A
I	R	L	E	U	B	T
Y	F	X	O	X	R	Z
1	2	3	4	5	6	7

2nd Dec.

A	S	H	I	P	I	N
H	A	R	B	O	U	R
I	S	S	A	F	E	B
U	T	T	H	A	T	I
S	N	O	T	W	H	A
T	S	H	I	P	S	A
R	E	B	U	I	L	T
F	O	R	X	Y	Z	Z
2	4	6	5	1	3	7